

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



用 Python 實作 CART 及 Random Forest



Johnny Chuang · [Follow](#)

12 min read · Feb 11, 2021



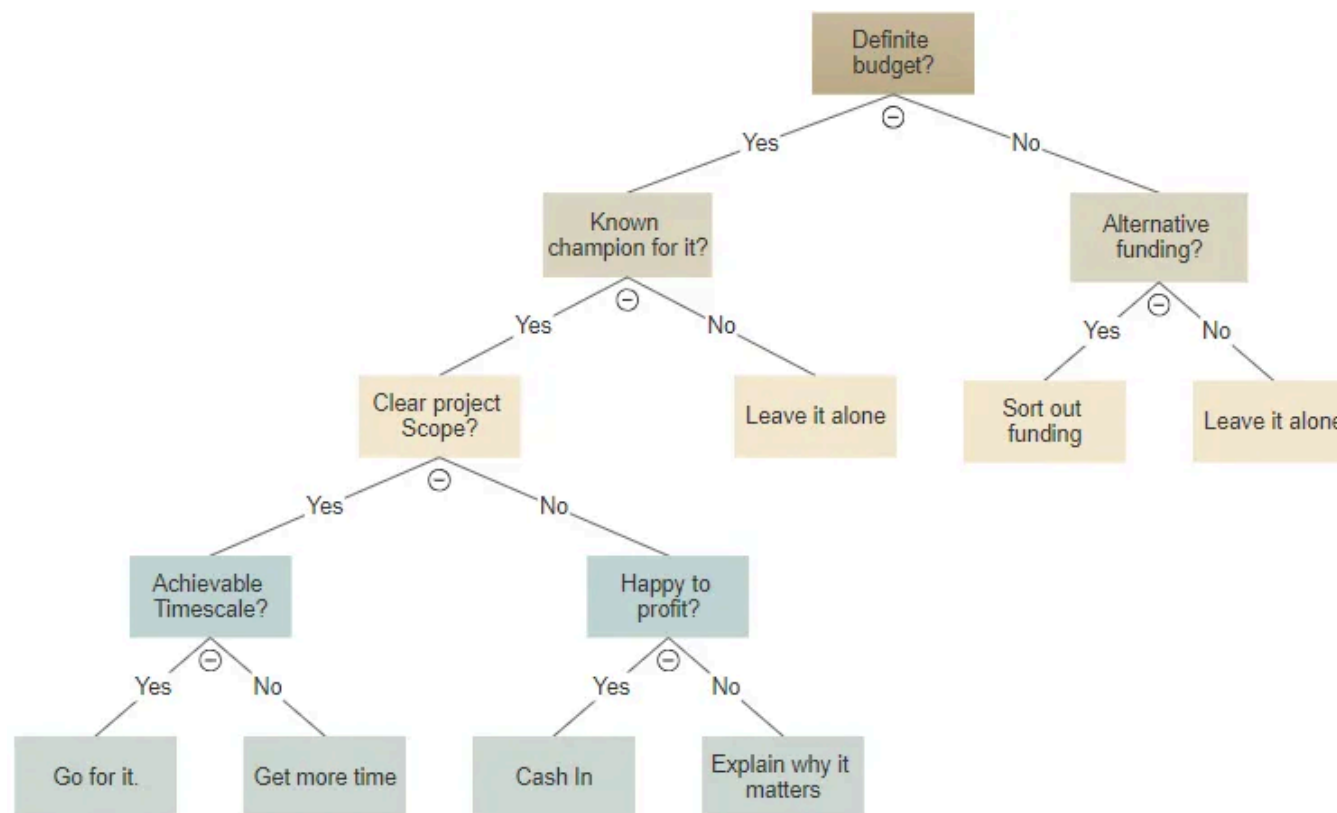
69



決策樹與 CART 演算法

Classification And Regression Tree(CART) 是一種經典的決策樹演算法，決策樹 (Decision Tree) 以用樹狀的方式往下建立子樹，每一個節點 (node) 分別為一和特徵相關的條件，而在決策樹中，歸屬在同一個判別條件下的資料點都會被給予相同的值，即發生機率皆為相同，因此決策樹的準確度並不會是最高，但因為其具有很高的可解釋性(看到切分的條件便可知道其意涵)，在商業世界中有很高的應用價值，故仍為機器學習中非常熱門的模型。而 CART 顧名思

義，就是一個能夠建立分類樹以及迴歸樹的決策樹演算法。CART 主要特性是建構的皆為二元樹，並針對分類、迴歸型問題選擇不同的損失函數。其重要假設會在下方提到。



An example of a decision tree

關於決策樹在機器學習上的應用，我們可以想成利用資料的各種特徵去當作判斷基準，透過計算用該特徵中不同的切分點所切出的類別的純度來決定判別條

件或是門檻值，並用預測出來的值和實際資料的值做比較，建出 **error** 最小，也就是預測能力最強的一棵樹來當作最後回傳的模型。

舉最常見的鳶尾花問題來說，我們可以先透過花萼長度選出最佳門檻值，從這樣的門檻值往下切分出不同的子樹，再從這些子樹往下蔓延，找出其他特徵的最佳門檻值，最後回傳最後節點中資料所屬類別中最大宗的那個作為符合所有門檻條件的預測類別。

	Sepal length	Sepal width	Petal length	Petal width	Class
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
⋮	⋮	⋮	⋮	⋮	⋮
150	5.9	3.0	5.1	1.8	virginica

Some rows of the Iris data set

CART 的基本假設

上面提到 CART 演算法有一些特別的假設，而分類樹和迴歸樹在處理於何處切分資料以及衡量純度的方式會有些許不同，由於實作部分將以分類樹為例，以下亦將著重在分類樹的假設。

1. 所有的樹皆為二元樹

2. 使用 Gini index 衡量資料不純度

$$1 - \sum_{k=1}^K \left(\frac{\sum_{n=1}^N \mathbb{I}[y_n = k]}{N} \right)^2$$

Formula for Gini index

Gini index 公式的概念是，用 1 減去每筆資料屬於第 k 個類別的機率平方總和，算出的便是整個資料的不純度。舉例來說，假設今天想做三個類別 (A,B,C) 的分類，當我們跑到某個節點，剩下 100 筆資料，而某一特徵中，透過 10 這個門檻值來切分資料會將資料切為 (40,60)，在這 40 筆資料當中，分別有 (10,10,20) 屬於 (A,B,C) 類別，在 60 筆資料中，分別有 (30,30,0) 屬於 (A,B,C)。

則這 40 筆資料的 Gini index = $1 - [(10/40)^2 + (10/40)^2 + (20/40)^2] = 0.625$

60 筆資料的 Gini index = $1 - [(30/60)^2 + (30/60)^2] = 0.5$

可以發現，**Gini index 越低代表資料純度越高**，在之後實作，我們將透過比較不同切法所切出的兩個資料集的 Gini 加總，來找出最低 Gini 的切法做為最佳門檻值。

3. 使用 **decision stump** 切分資料

Decision stump 其實就是單層的決策樹，我們可以將它想像在一條一維的線上嘗試將資料切分成兩部分的一個演算法，上面提到針對每一個特徵去找出最佳門檻值就是 decision stump 的體現。而這樣一刀切的分法讓 decision stump 本身只能稱為一個 weak learner，但它仍不失為一個好的 base algorithm。

4. 若終止條件達成，該節點回傳使 **in-sample error** 最小的常數

5. 終止條件：**impurity(不純度)**為 0，或節點中所有資料皆相同

第一個終止條件，也就是不純度為 0 時，代表此節點所需要切分的資料已經全部都屬於同個類別，因此不需再做切分。第二個終止條件也非常直觀，若所有資料都相同，則沒有 decision stump 可以進行。

CART 演算法實作

原始碼及資料：<https://github.com/Johnnyyy24/CART-RandomForest>

這次使用的是林軒田老師於機器學習技法中所提供的資料集，分別分成 **training data** 和 **testing data** 兩個檔案，有興趣的人可以直接把整包內容下載下來。以下會進程式碼內每一步驟的說明。

首先，先將所有需要用到的 **packages import** 進來。

```
1 import pandas as pd
2 from operator import itemgetter
3 import copy
4 import random
5 import statistics
```

package.py hosted with ❤ by GitHub

[view raw](#)

0	3.956323	1.801302	-0.560404	-0.399497	0.416019	-1.019041	1.085559	-0.388153	1.156472	-0.024579	1
1	1.323972	-2.401976	2.004373	-0.189241	-0.113315	0.447637	-0.213228	0.014771	-0.665521	-0.263573	1
2	1.412873	2.517723	1.815558	-2.062033	-1.619071	-0.205693	-0.753901	-0.858209	-0.132313	-0.413573	-1
3	4.117572	0.748632	0.147626	-0.638341	-0.874174	-0.936210	0.746794	-0.176653	0.426626	0.536221	1
4	-3.276037	2.048978	-0.547063	-0.396693	0.030872	-1.037922	0.173414	-0.207210	-1.129323	0.069900	1
...	...	...	...	...	...	...	...	...	...	...	...
995	1.646005	-2.304461	-0.685875	-1.242158	0.089632	0.414168	0.032295	-0.277479	-0.577471	0.114737	1
996	2.824186	0.051084	-1.219477	-1.533916	-1.158722	-0.876487	-0.248153	0.328429	0.052668	-0.394568	1
997	-3.542418	1.909270	0.030633	-0.830695	-0.063975	-0.573080	-0.497818	-0.572063	-0.699575	0.195609	1
998	1.468522	-2.359230	-0.242721	-0.793927	0.992404	0.309329	0.954874	1.048745	-0.574783	0.236064	1
999	3.454670	-1.043283	1.874078	-0.719441	-1.042442	-0.528895	-0.228148	0.028543	-0.517472	1.126000	1

1000 rows × 11 columns

training data

查看資料發現，training data 和 testing data 很單純，不需要進行額外編碼，兩者都分別有 1000 筆資料，而第 1-10 行為 feature 的實際數值，第 11 行則為分類結果 (label)。先幫資料加上 column name，並把資料轉成 list of list 的型態讓後面的運算更方便。

```
1  columns_name = list()
2  for i in range(1,11):
3      columns_name.append("x"+str(i))
4  columns_name.append("y")
5
6  train_data = pd.read_excel('train.xlsx',names=columns_name, header=None)
7  test_data = pd.read_excel('test.xlsx',names=columns_name, header=None)
8
9  def TransformToList(data): #把資料轉成 list 呈現
10     data_list = list()
11     for i in range(len(data)):
12         data_list.append(list(data.iloc[i]))
13     return data_list
14
15  train_list = TransformToList(train_data)
16  test_list = TransformToList(test_data)
```

preprocessing.py hosted with ❤ by GitHub

[view raw](#)

接下來就進到主程式部分，我們先宣告一些函式讓整個主程式的可讀性提高。首先，*get_thresholds* 幫助我們完成在每個特徵下列出所有可能的門檻值。傳進 *get_thresholds* 的資料必須先排序好，我們由小到大選擇每兩個特徵值之間的平均數當作一個可能的門檻值，再加入比最小特徵值小，以及比最大特徵值大的兩個門檻值就完成在該特徵中所有門檻值。若 *dataset* 有 N 筆資料，則回傳的 *list* 中最多可能有 $N+1$ 個門檻值。

data_split 則幫助我們把小於門檻值的所有資料分到左子樹，大於等於門檻值的則是右子樹，回傳的資料型態為兩個代表左、右子樹資料的 *list*。


```
1 def get_thresholds(index,dataset): #dataset sent in here should be already sorted
2     thresholds = []
3     thresholds.append(dataset[0][index]-1) #first threshold
4     for i in range(len(dataset)):
5         if(i!=0): #if not the first row
6             prev = dataset[i-1][index]
7             cur = dataset[i][index]
8             if(prev!=cur):
9                 thresholds.append((prev+cur)/2) #add the mean as one threshold
10    thresholds.append(dataset[-1][index]+1) #last threshold
11    return thresholds
12
13 def data_split(index, value, dataset): #把資料分成左子樹・右子樹
14     left,right = list(),list()
15     for row in dataset:
16         if(row[index] < value):
17             left.append(row)
18         else:
19             right.append(row)
20     return left, right
```

threshold&data_split.py hosted with ❤ by GitHub

[view raw](#)

gini_index 計算透過某一門檻值分出的兩個子樹加總的 *gini* 值，詳細公式和上面提到的一樣，不一樣的部分是我們把算出來的 *gini index* 乘上了該子樹的資料長度，原因是一個較多資料的子樹的純度對我們分類是更重要的，因此透過考慮長度來給予權重。*data_same* 則是為了判斷終止條件，上面提到終止條件有兩個，分別是不純度已為 0，以及子樹中的資料皆相同，此處考慮的就是第二個終止條件。

```

1  def gini_index(groups): #return the best threshold for that index
2      gini = 0.0 #of the threshold
3      for group in groups:
4          if(len(group)==0): #avoid divided by zero
5              continue
6          pos_y = 0 #how many positive labels in the group
7          for row in group:
8              if(row[-1]==1):
9                  pos_y += 1
10         neg_y = len(group) - pos_y
11         gini_index = (1 - (pos_y/len(group))**2 - (neg_y/len(group))**2)
12         gini += len(group) * gini_index #to weight different lengths differently
13     return gini
14
15 def data_same(original_dataset):
16     dataset = copy.deepcopy(original_dataset)
17     for row in dataset:
18         row.pop() #把label拿掉
19     allsame = True
20     for i in range(len(dataset)):
21         if(dataset[0]!=dataset[i]):
22             allsame = False
23     return allsame

```

gini&data_same.py hosted with ❤ by GitHub

[view raw](#)

再來宣告 class 來當作儲存節點的資料結構。當一個新的節點被宣告，我們至少需要 *index* (第幾個特徵)、*threshold* (該特徵的門檻值)來當作節點的 *attributes*。而 *left* 和 *right* 被初始為 *None*，代表一開始預設每個節點都是沒有 *child node* 的，資料切分完我們才會把新的節點接在其之下(*set_left*,

set_right)。 *label_l* 和 *label_r* 則是儲存樹的架構下最下方節點中小於和大於特徵門檻值的分類結果，我們透過 *set_label* 來達成資料中的多數決，舉例來說，若是小於特徵門檻值的資料中屬於 1 的資料量較多，往後當有資料滿足所有條件走到這個節點，我們的演算法就會自動將其分類為 1 這個類別。

```
1  class Node:
2      def __init__(self, index, threshold):
3          self.left = None
4          self.right = None
5          self.index = index
6          self.label_l = 0
7          self.label_r = 0
8          self.threshold = threshold
9      def set_left(self, aNode):
10         self.left = aNode
11      def set_right(self, aNode):
12         self.right = aNode
13      def set_label(self, best_groups):
14         if(self.left == None and self.right == None):#沒有子節點的代表是最下方的節點，標示出他們的label
15             l_sum, r_sum = 0, 0
16
17             for row in best_groups[0]: #左節點
18                 l_sum += row[-1]
19             if(l_sum>0): #如果+1比-1還要多的話，標示為+1(投票機制)
20                 self.label_l = 1
21             else:
22                 self.label_l = -1
23
24             for row in best_groups[1]: #右節點
25                 r_sum += row[-1]
26             if(r_sum>0): #如果+1比-1還要多的話，標示為+1(投票機制)
27                 self.label_r = 1
28             else:
29                 self.label_r = -1
```



正式進入主程式的部分，一開始先將變數設定成極大的值，然後開始跑每個 `index`，計算其每個特徵中不同門檻值的 `gini` 值，如果新算出來的 `gini` 值比我們目前跑出的 `best_gini` 更小就取代它，並用變數記錄下來。最後當所有特徵門檻值都考慮過後，我們建立一個新的節點來儲存這些變數。如上述，考慮完所有門檻值後我們還要考慮資料是否切分乾淨了，我們用一個變數 `terminal` 來記錄終止條件是否成立。若終止條件成立，代表我們已經將資料切分完整，於是透過上述提到的 `set_label` 儲存學習到的分類方式並回傳該節點。若終止條件不成立，我們把切出來的兩份資料分別再丟回該節點的左、右節點，透過遞迴的方式重複上述步驟將整棵樹建構出來。

```
1 def get_split(dataset):
2     best_index, best_threshold, best_gini, best_groups = 9999, 9999, 9999, None
3     terminal = False #判斷是否終止
4     for index in range(10):
5         sorted_data = sorted(dataset, key=itemgetter(index))
6         thresholds = get_thresholds(index,sorted_data)
7         for threshold in thresholds:
8             groups = data_split(index,threshold,sorted_data)
9             gini = gini_index(groups)
10            if(gini<best_gini):
11                best_gini = gini
12                best_index,best_threshold,best_gini,best_groups = index,threshold,gini,groups
13    aNode = Node(best_index,best_threshold)
14
15    if(best_gini==0 or data_same(dataset)): #終止條件判斷
16        terminal = True
17
18    if(terminal):
19        aNode.set_label(best_groups)
20        return aNode
21    else:
22        aNode.set_left(get_split(best_groups[0]))
23        aNode.set_right(get_split(best_groups[1]))
24        return aNode
```

get_split.py hosted with ❤ by GitHub

[view raw](#)

整理一下現在的進度，跑完上述的程式後，我們得到了最終回傳的節點，**其實就是一顆完整的樹**，因為這個節點串接著它的子節點，而這些子節點又將繼續延伸下去。接著我們建立一個函式 *prediction* 來幫助我們衡量這個演算法的

error。在 *prediction* 中，由上往下遍歷每一個節點，並透過節點儲存的切分條件來切分資料，直到走到最下方的節點(即沒有子節點的節點)我們才開始計算 **error**。*results* 這個變數則是儲存多個(資料,預測值)的 **tuple**，在之後的隨機森林衡量 **error** 時才會派上用場。在這個函式中我們一樣利用遞迴的方式去遍歷並完成 **error measurement**。

```
1 def prediction(dataset,aNode):
2     index, threshold = aNode.index, aNode.threshold
3     #先依index排序·再split成兩組
4     sorted_data = sorted(dataset, key=itemgetter(index))
5     groups = data_split(index,threshold,dataset)
6     err = 0
7     results = []
8     if(aNode.left == None): # it doesn't have any child node, we should do the classification
9         #左節點
10        for row in groups[0]:
11            results.append((row,aNode.label_l))
12            if(row[-1]!=aNode.label_l):
13                err += 1
14        #右節點
15        for row in groups[1]:
16            results.append((row,aNode.label_r))
17            if(row[-1]!=aNode.label_r):
18                err += 1
19
20        return err,results
21
22    else: # the node has child nodes, keep traversing
23        err_l,results_l = prediction(groups[0],aNode.left)
24        err_r,results_r = prediction(groups[1],aNode.right)
25        err += err_l+err_r #add the classification error of two child nodes to err
26
27        for element in results_l:
28            results.append(element)
29        for element in results_r:
30            results.append(element)
31        return err,results
```

prediction.py hosted with ❤ by GitHub

[view raw](#)

以上，CART 演算法的部分就完成了，我們可以簡單透過前面定義過的函式計算出 in-sample error (以下簡稱 Ein) 以及 out-of-sample error (以下簡稱 Eout) 分別為 0% 和 16.6%，敏銳的大家應該會察覺到 0% 的 Ein 事有蹊翹，這代表我們把資料學得相當完全，最後的子節點們成功把 training data 完全切分開來了，這會衍生一些值得討論的問題，在下方會提及到。

```
[7] aNode = get_split(train_list)
     err,result = prediction(train_list,aNode)
     print(err/len(train_list))
```

0.0

```
[8] aNode = get_split(train_list)
     err,result = prediction(test_list,aNode)
     print(err/len(train_list))
```

0.166

CART 與 Random Forest

前面完成 CART 的實作後，衡量 error 時我們發現我們做出來的 Ein 竟然為 0，這樣過於美好的結果其實暗示著我們的演算法 overfit 了，即我們將資料學的太完全，就好像一個把整本教科書都背起來卻沒有融會貫通的學生，在實際碰到沒看過的題型時表現可能不會太好。要想改善這樣的結果，我們至少有兩種方式能達成：

Tree Pruning

CART 會 overfit 是因為其貪婪演算法的性質，會只在乎當期的短期利益不顧後果地分枝生長，因此我們可以透過加入節點數的懲罰項來限制節點數，或是從一棵完全長成的樹由下往上剪枝來達成 tree pruning。

Bagging(Bootstrap Aggregating)

Bagging 則是我們採用的方法。Bagging 是 Breiman 於 1996 年提出的方法，其概念是從訓練資料中透過抽樣後放回的方式創造多組具備隨機性的資料，因為是抽樣後放回會導致抽取出的樣本之間有一些資料重複，但因為整體樣本不同，透過這些不同樣本建立出的分類器也不同，我們再用所有分類器的結果來進行多數決投票，將一堆弱學習器結合，建構出一個更強的模型，這就是 ensemble learning 的體現。

其實，以上就是隨機森林的概念。森林指的是很多棵樹，隨機則是我們在 bagging 所做的事，因此可以得到一個結論：

Random Forest = Bagging + Fully-grown CART decision tree

看到這大家大概可以猜到，完成 CART 後隨機森林的實作其實非常簡單，我們已經知道如何用完整的 training data 建出一個 CART，現在我們只要把 Bagging 實作完，用每一次 Bagging 得到的資料去訓練出一個新的 CART，最後再將所有 CART 的分類結果用投票的方式做最後整合，我們的隨機森林演算法就完成了！

Random Forest 演算法實作

首先宣告 *Bagging* 這個函式，傳入的變數包含我們希望抽樣多少比例的資料，以及資料本身。透過 *random* 內建的 *function* 得到隨機抽樣得到的 *index* 後，我們將這些 *index* 對應的資料放進 *bagging_data* 中並將其回傳。

再來是 *RandomForest* 函式，傳入資料和我們希望用多少棵樹來建構森林後，每一個 *iteration* 會建立出一棵樹，我們將這些樹的預測結果儲存下來，最後將這些分類結果相加得到一個整數，該整數如果大於 0 代表較多棵樹判斷為 +1，反過來說則是較多棵樹認為是 -1。透過將這樣的分類結果相加後，我們可以得到在每個資料上實際的 *label*，以及隨機森林多數決的結果。

```

1  def Bagging(fraction,dataset):
2      bagging_len = int(fraction*len(dataset))
3      bagging_data = []
4      index_list = random.choices(range(0,len(dataset)),k=bagging_len)
5      for index in index_list:
6          bagging_data.append(dataset[index])
7      return bagging_data
8
9  def RandomForest(training_data,testing_data,NumberofTrees): #隨機森林
10     errs = []
11     results = {}
12     for i in range(NumberofTrees): #the bagging process
13         temp_train = Bagging(0.5,training_data)
14         aNode = get_split(temp_train)
15         err,result = prediction(testing_data,aNode)
16         errs.append(err)
17         for row in result:
18             if(tuple(row[0]) not in results): #A list cannot be the key to a dict, so tranform th
19                 results[tuple(row[0])] = row[1]
20             else:
21                 results[tuple(row[0])] += row[1] #to add them up means to vote, for instance if s
22     return statistics.mean(errs)/len(testing_data), results

```

最後兩個 function 相對簡單，*sign* 判斷上述提到多數決的結果為何，*ErrorFunction* 則是透過比較每一筆資料最後一個元素 (label) 和多數決的結果來判斷是否犯錯。

```
1 def sign(x):
2     if(x>0):
3         return 1
4     else:
5         return -1
6 def ErrorFunction(results):
7     error = 0
8     for key in results:
9         if(sign(results[key])!=key[-1]):
10             error += 1
11     return error/len(results)
```

sign&ErrorFunction.py hosted with ❤ by GitHub

[view raw](#)

我們可以再宣告另一個 OOB 版本的 Random Forest，OOB(Out-of-bag)的概念是先用抽樣建立模型時沒有被抽到的那些資料來當作衡量錯誤的標準，如此以來我們甚至不用用到測試資料就能得到一個模型的泛化誤差估計值，相當節約環保。我們只需要在原本的 *RandomForest* 中多加一段選出 *oob_data* 的部分就可以完成這個版本的隨機森林。

```
1 def RandomForest_oob(training_data, NumberofTrees):
2     errs = []
3     results = {}
4     for i in range(NumberofTrees): #the bagging process
5         oob_data = []
6         temp_train = Bagging(0.5, training_data)
7         aNode = get_split(temp_train)
8         #find oob data
9         for row in training_data:
10             if(row not in temp_train):
11                 oob_data.append(row)
12             err, result = prediction(oob_data, aNode)
13             errs.append(err)
14             for row in result:
15                 if(tuple(row[0]) not in results):
16                     results[tuple(row[0])] = row[1]
17                 else:
18                     results[tuple(row[0])] += row[1]
19     return statistics.mean(errs)/len(training_data), results
```

rf_oob.py hosted with ❤ by GitHub

[view raw](#)

以 200 棵樹構成的隨機森林為例，用 *ErrorFunction* 算出來的 E_{in} 為 1.3%， E_{out} 為 15.2%，而 E_{oob} 為 6.9%。可以發現 E_{in} 雖然增加，但 E_{out} 降低了 1.4%，這也是我們希望達成的結果。讓我們將思路想回學生的例子：從老師的角度出發，比起學生在課本習題中每題都學到完美，但實際考試時表現不怎麼樣，我們更希望的是學生在習題上能夠犯點錯，但碰到考試時表現也能不錯。

```
[14] err, results = RandomForest(train_list, train_list, 200)
      print(ErrorFunction(results))
```

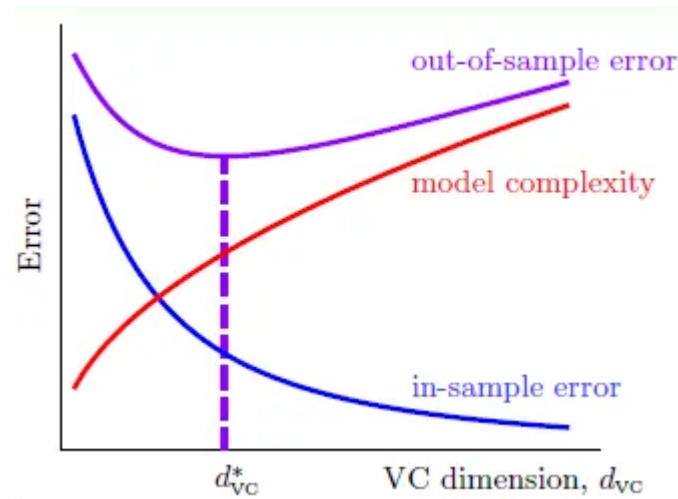
0.013

```
[17] err, results = RandomForest(train_list, test_list, 200)
      print(ErrorFunction(results))
```

0.152

```
[11] err_oob, results_oob = RandomForest_oob(train_list, 200)
      ErrorFunction(results_oob)
```

0.069



最後這是機器學習基石當中其中一頁投影片的內容，這張圖資訊量非常龐大，但簡單地套回我們這一路上的發現，我們從一棵 CART 走到一片隨機森林的路上，其實正是降低 **model complexity**，讓我們更接近 **optimal VC dimension** 的過程。建樹也建林，我們能夠免於見樹不見林的危險。

這篇文章就到這裡結束了！如果有誤或是有想討論的部分，非常歡迎留言或是 聯絡我，感謝！

References

Breiman, L. (1996a). Bagging predictors. Machine Learning 26(2), 123–140.

Breiman, L. (2001) Random Forests

Hsuan-Tien, Lin. Machine Learning Foundations, Machine Learning Techniques

Machine Learning

Random Forest

決策樹

機器學習

隨機森林



Written by Johnny Chuang

233 Followers · 102 Following

Follow

NTU Undergrad | DeFi Enthusiast <https://defi.substack.com/>

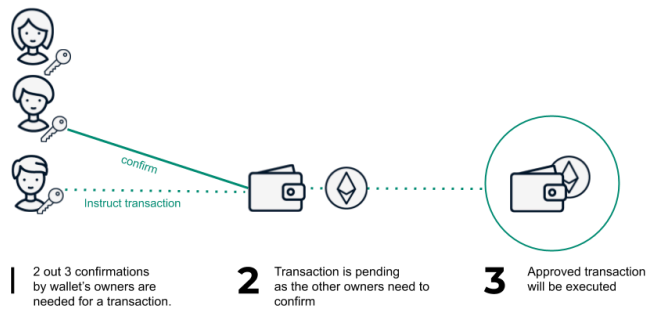
No responses yet



Jumbo Hsieh

What are your thoughts?

More from Johnny Chuang

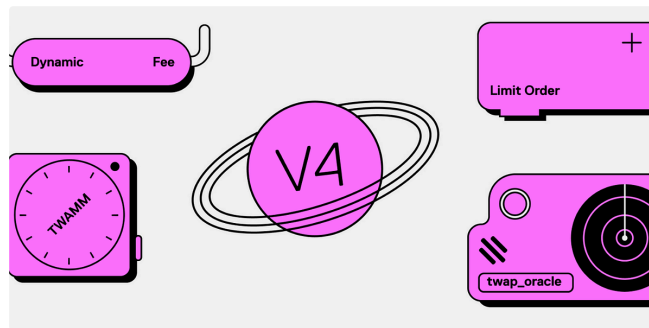


Johnny Chuang

Gnosis Safe 多簽錢包使用教學

過去兩週相信對許多人來說都相當不好受，FTX 的崩塌以及其伴隨而來各種造市商、中...

Nov 20, 2022 21

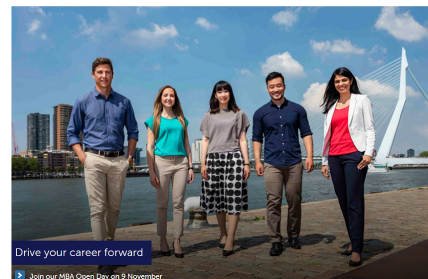


Johnny Chuang

Uniswap V4 Hooks 將帶來什麼革新？

為什麼我們需要 Uniswap V4 ？

RSM Rotterdam School of Management
Erasmus University



Johnny Chuang

我從大數據學到的事(2) A/B Testing

前言

Nov 27, 2019 67



Johnny Chuang

【AppWorks 實習心得】不要放棄問為什麼的權利

Jun 24, 2023  119



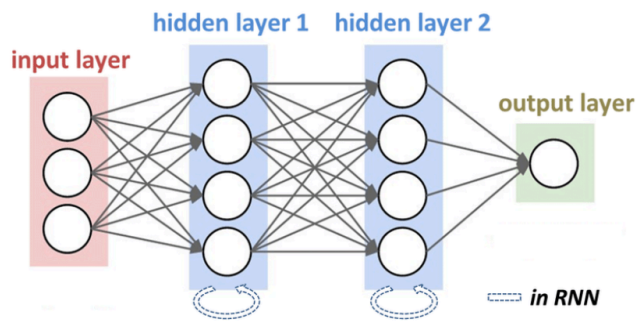
在今年六月底完成了在 AppWorks 為期四個月的實習。回頭看，從大一第一次聽到...

Jul 19, 2020  182

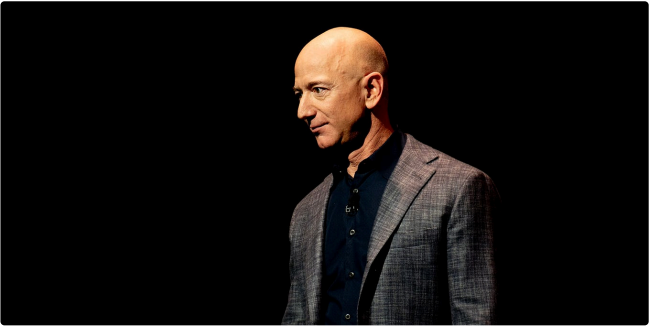


See all from Johnny Chuang

Recommended from Medium



r



Jessica Stillman

A Comprehensive Guide to Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are a class of neural networks designed to handle...

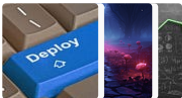
Nov 6, 2024 10

Jeff Bezos Says the 1-Hour Rule Makes Him Smarter. New...

Jeff Bezos's morning routine has long included the one-hour rule. New...

Oct 30, 2024 24K 730

Lists



Predictive Modeling w/ Python
20 stories · 1853 saves



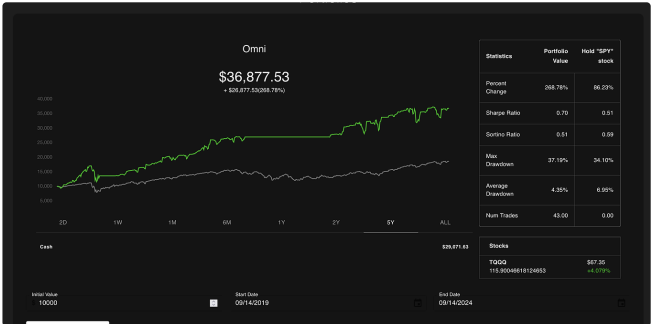
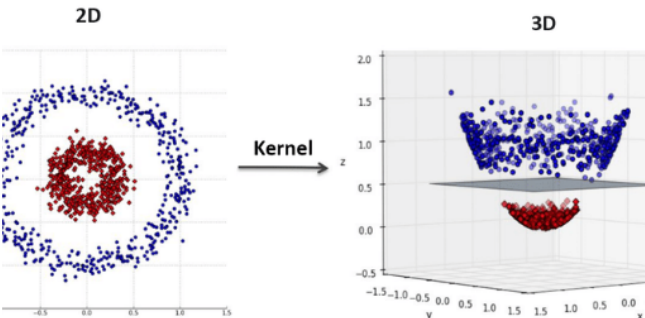
Practical Guides to Machine Learning
10 stories · 2221 saves



Natural Language Processing
1973 stories · 1617 saves



The New Chatbots: ChatGPT, Bard, and Beyond
12 stories · 562 saves

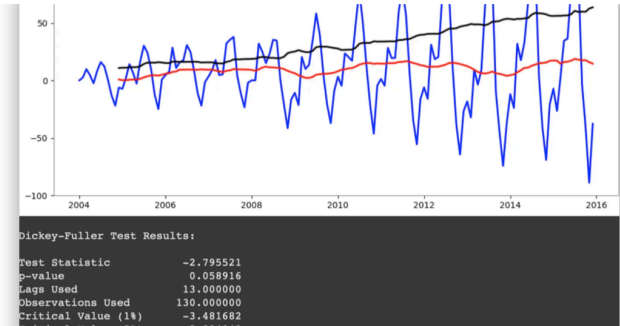


 Abhishek Jain

SVM kernels and its type

Support Vector Machines (SVMs) are a popular and powerful class of machine...

Sep 12, 2024  69



 In Data Science Collective by  panData

Hands-on: Time Series Forecasting

From Data Preparation to Stationarity and Predictive Modeling

★ 3d ago  81  3

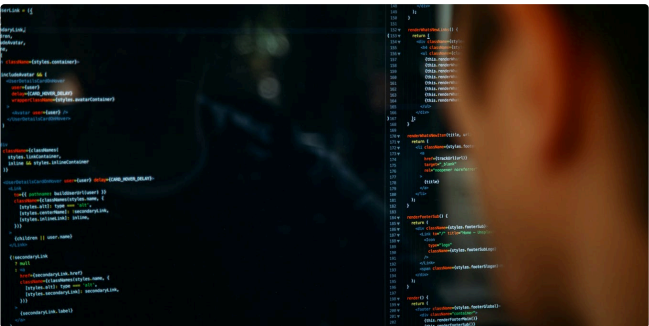


 In DataDrivenInvestor by Austin Starks

I used OpenAI's o1 model to develop a trading strategy. It is...

It literally took one try. I was shocked.

★ Sep 16, 2024  9.1K  239

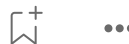


 In GoPenAI by agus abdul rahman

SHAP for Machine Learning: A Step-by-Step Python Tutorial

Learn how to interpret machine learning models using SHAP values with hands-on...

★ Feb 26  114



See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Terms](#) [Text to speech](#) [Teams](#)